

COMPARING WILDFIRE RISK PREDICTION USING ML-BASED MODELS

[GitHub Repository](#)

Ella Chee, Khushi Khan, Willbert Clement Christianto
Khoury College of Computer Sciences Northeastern University
April 16, 2026

I. INTRODUCTION

Climate change has significantly increased wildfires across the globe, making it a rising priority to accurately identify areas at risk of wildfire activity. Existing work has shown that ML (ML) methods, specifically convolutional neural networks (CNNs) and Bayesian models, are effective at this task. Multi-layer perceptrons (MLPs) remain a foundational architecture and a performance ceiling on high-dimensional image classification tasks, despite being less thoroughly benchmarked in literature.

The goal of this project is to compare the performance of an MLP, CNN, and Bayesian logistic regression model and assess strengths of each approach. The dataset is the *Wildfire Prediction Dataset (Satellite Images)* on [Kaggle](#), which contains 42,850 labeled satellite images of areas that have previously experienced wildfires in Canada.

II. LITERATURE REVIEW

A. Multi-Layer Perceptrons

Jain et al. [3] reviews many ML applications in wildfire science and management, synthesizing ~300 publications including fire detection, susceptibility mapping, and fire behavior prediction. The methods include random forests, support vector machines, artificial neural networks, and deep learning architectures. MLPs have been at the forefront of this solution since the 1990s, serving as benchmark classifiers in the above tasks. However, there is noticeable outperformance by CNNs when spatial image data is involved, as MLPs are not able to exploit local spatial correlations necessary in geographic imagery, since MLPs can only process flattened input vectors. Despite rapid growth in satellite remote sensing datasets, deep learning applications to large-scale satellite image classification for wildfire prediction remain relatively underexplored relative to tabular environmental feature approaches. This paves the way for our image-based MLP experiment with binary classification of satellite wildfire imagery.

Bjånes et al. [4] evaluates a deep learning model for wildfire susceptibility mapping with satellite images. Working with fire occurrence records from regions in Chile spanning 2013 to 2019, the authors analyze various factors contributing to wildfires (i.e. elevation, slope, temperature, wind speed, precipitation, land cover type, distance to roads and urban areas, etc.). All five models (MLP, two CNN variants, a multi-input deep network, XGBoost, and proposed ensemble of the two best-performing CNN) take 25x25px image patches as inputs. Each model's predictive capabilities are evaluated based on AUC, accuracy, sensitivity, and F1 score. The ensemble model achieved the highest performance with an AUC of 0.953, followed by CNN-1 with an AUC of 0.902. The MLP was consistently outperformed across all metrics by both CNN architectures. The study also finds that sample patch size influences performance. This closely parallels the present study, which applies an MLP to 350x350px satellite images of Canadian wildfire-affected regions, and provides a similar empirical benchmark for understanding the performance ceiling of spatially-unaware feedforward networks applied to satellite wildfire classification.

B. Convolutional Neural Networks

Kanwal et al. [5] explore how a CNN can learn to detect patterns associated with fire occurrence and spread in satellite imagery. In particular, CNNs can identify spatial features from images to distinguish between wildfire-affected and unaffected regions more effectively than traditional methods. This approach demonstrates the strength of CNNs in extracting meaningful spatial patterns from large remote sensing datasets. In their experiments, models incorporating CNN-extracted features achieved prediction accuracies of around 96%, outperforming several traditional ML approaches. Their work suggests that deep learning models can automate wildfire detection by learning visual indicators of fire activity directly from satellite imagery. These findings support the use of CNNs as a foundation for wildfire prediction models that rely on satellite-based observations.

Marjani et al. [6] propose a hybrid deep learning model that combines CNNs with bidirectional long short-term memory (BiLSTM) networks to predict wildfire spread over time. Similar to the

approach by Kanwal et al. [5], the CNN analyzed spatial features from satellite imagery. This method was expanded further by integrating a BiLSTM component that analyzes temporal patterns to model wildfire progression over time. By combining spatial and temporal information, the model better captures how wildfire conditions evolve across consecutive time periods. Their results show that the CNN-BiLSTM model achieved prediction accuracies above 90% and outperformed baseline CNN and LSTM models. This demonstrates that incorporating temporal dependencies can significantly improve predictive performance in wildfire modeling tasks and highlights the potential benefit of integrating temporal or environmental data when developing wildfire prediction systems.

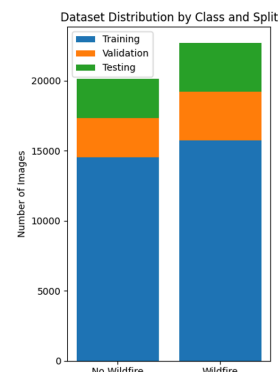
C. Bayesian Models

Charizanos and Demirhan [1] predicted wildfire event probability using Bayesian ML from environmental predictors and forest vulnerability. Spectroscopic analysis of Australian forest satellite image data from Google Earth Engine was used to measure forest vulnerability as a Normalized Difference Vegetation Index. Other predictors (temperature, wind speed, and humidity) were drawn from wildfire reports for consideration with a multivariate normal prior distribution. Bayesian logistic regression with a Bernoulli likelihood function was used to predict the probability of a wildfire event associated with given predictors and forest vulnerability index. Incorporating hierarchical Bayesian logistic regression with multivariate normal prior distribution helped to address collinearity, while Bayesian model averaging helped counter uncertainty from multiple models with combinations of predictors. Overall, Bayesian hierarchical logistic regression with noninformative priors predicted low probabilities during winter/autumn conditions (<6%), 31.5% in summer, and 64.6% during extreme conditions. Accounting for informative multivariate prior distributions had similar probabilities but pushed the wildfire likelihood to 89.6% for extreme summer conditions. Across both modeling, forest vulnerability was the dominant predictor, ultimately with a high accuracy of 96.9%.

Dlamini [2] also examines fire risk prediction, though instead by implementing Bayesian networks with geographic information systems (GIS) and remote sensing data across Swaziland. The study integrated several more factors: 13 total biophysical (altitude, slope, land cover, humidity, etc.) and socioeconomic (road density, human population density, etc.) extracted from satellite imagery. Fire location data was drawn from Moderate Resolution Imaging Spectroradiometer Terra and Aqua daily active fire data. Images were converted into a GIS “case file” where each pixel is associated with its specific environmental and fire state. The fire node of the Bayesian network was set to a beta distribution, while maximum likelihood estimates were computed with the Expectation-Maximization algorithm to comprise Conditional Probability Tables (CPTs) for each node, representing probabilistic relationships. These CPTs were reimported to GIS to generate fire risk maps categorizing areas into “very low” to “extreme” risk zones, with over 90% accuracy for predicting active fire (93.14%) and burnt areas (96.64%).

III. DATA COLLECTION AND METHODOLOGY

The dataset is sourced from Kaggle allowing for binary classification of wildfire risk using real satellite images from Canada, labeled by regions that have and haven’t experienced wildfire. It was constructed by mapping wildfire event coordinates (based on areas with recorded fire activity) and extracting geographic/environmental relevance. To prepare the data for modeling, images were resized, normalized, then converted into numeric arrays: flattened for the MLP and across RGB channels to support probabilistic inputs for Bayesian analysis. The predefined train, validation, and test splits (70%, 15%, 15%) facilitated consistent evaluation across all approaches, ensuring that the data aligns with both deterministic and uncertainty-aware modeling objectives.



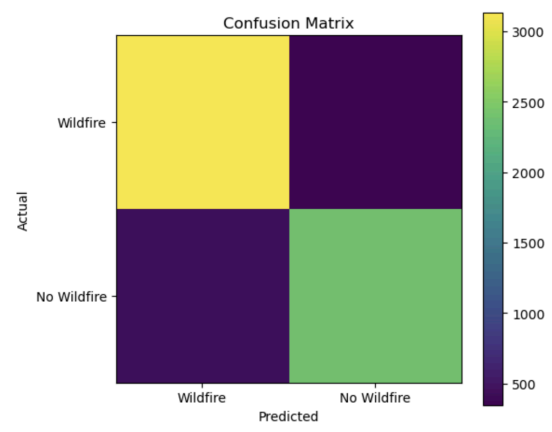
- *MLP*: The MLP is implemented manually in Python. The model maps an input vector (flattened image pixels) through a hidden layer (ReLU), and generates an output probability (Sigmoid). It's trained using binary cross-entropy loss and optimized by gradient descent. This framework uses existing class resources, but carries out several adjustments to handle high-dimensional image data more efficiently. First, mini-batch gradient descent was used instead of full-batch updates to improve computational stability. The initialization technique was applied to the weights to prevent exploding gradients with ReLU activations, and input features were normalized using z-scoring to ensure consistent scaling. Lastly, vectorized operations are used to clip sigmoid inputs for numerical stability. These deviations extend the basic MLP framework from class to better suit large-scale image classification.
- *CNN*: The CNN is implemented using *Keras* and *Tensorflow* in Python. The model passes normalized data through a convolutional layer that applies ReLU, outputs 16 channels, then max pools the result with a (2,2) kernel. The second convolutional layer applies a ReLU function to the first pooled layer, outputting 32 channels. Max pooling is applied once more on the output of the second convolutional layer with a (2,2) kernel size. Each convolutional layer utilizes a (3,3) kernel with a stride of 1. Then, the pooled layer is flattened and passed through a hidden layer that applies ReLU, outputting 64 units. Finally, an output layer is defined with a Sigmoid activation function and an output space of 1 unit. The model is trained using an Adam (Adaptive Moment Estimation) optimizer, which trains in small batches of 32 images over 10 epochs. This optimizer is efficient on large datasets and adjusts learning rates dynamically during training. Finally, binary cross-entropy is used to calculate loss, as this task has a binary outcome.
- *Bayesian*: The Bayesian logistic regression model is designed with *brms* in R, establishing a default prior with a Bernoulli distribution, and Bernoulli posterior, as is standard for binary classification. 350x350px images were resized to 16x16px, RGB values were collapsed into a single value per pixel, and finally flattened into a single vector such that each pixel represents an input feature (resulting in 256 total features). Training was set up with four chains, 10000 iterations, and 1800 burn-in iterations (18% of the initial samples).

IV. RESULTS AND INTERPRETATION

A. Multi-Layer Perceptrons

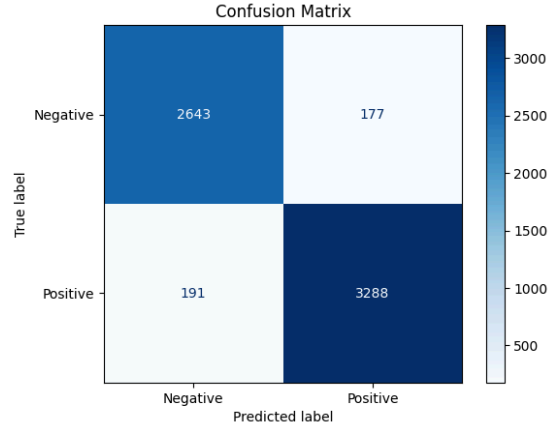
The MLP achieved stronger-than-anticipated results for predicting wildfire risk. The model had a test accuracy 90% and balanced performance across both classes. Additionally, there was an F1-score of 0.89 for wildfire and 0.91 for no wildfire, indicating that it is better at identifying wildfire-affected regions but the difference is immaterial to the scope/context of this project. Precision and recall are also closely aligned, indicating no major biases toward false positives or false negatives.

The confusion matrix further supports our thesis: a majority of predictions are accurate with relatively few misclassifications between classes. The mini-batch gradient descent and normalization techniques also support optimization stability that minimize overfitting. Overall, while the MLP lacks the ability to explicitly capture spatial structure, it still achieves strong predictive capabilities.



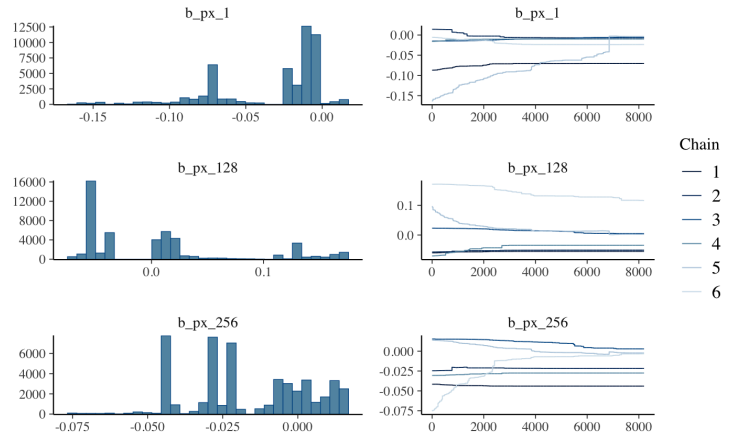
B. Convolutional Neural Networks

The CNN effectively captured spatial patterns in the satellite images. As shown by the confusion matrix, the model rarely predicted false negatives and false positives. The model achieved an accuracy, precision, F1 score, and recall of approximately 95% on the test dataset. The high precision indicates that the model is predicting wildfires accurately, while the high recall underlines that the model rarely labels images at risk of a wildfire as the latter class. Obtaining a high recall score is especially valuable in wildfire detection, as false negatives can have a detrimental impact to human life and the environment. The number of false negatives is lower than the number of false positives, which aligns well with this scenario. The F1 score further demonstrates a good balance between precision and recall, suggesting robust overall performance.



C. Bayesian

The Bayesian logistic regression failed to meaningfully represent the distinction between satellite images of areas with and without risk of wildfire. Decreasing the image size while increasing the number of iterations had some improvement, though various combinations of hyperparameters failed to reduce the r-hat to 1.00. In the final model, there were 2540 divergent transitions after warmup, with the largest r-hat being 7.52, indicated poor mixing of chains. As shown in the plots from a sample of features above, the posterior distribution fails to adequately mirror the Bernoulli, and rather than displaying random, hairy caterpillar-like chains across iterations, there seems to be little to no mixing of chains. Despite the poor convergence however, the model managed an overall accuracy of ~82%, and F1 score of 0.84, ranking slightly lower than the MLP.



V. CONCLUSION AND FUTURE WORK

Overall, both the non-probabilistic models performed effectively in detecting the likelihood of wildfires. The CNN achieved a test accuracy of 94%, with the MLP at 90%, while the Bayesian logistic regression underperformed most at 82%. This result was fairly expected. Though MLP fails to capture spatial relationships, the “deep” aspect of applying layers and activation functions makes up for it. The CNN takes this groundwork a step further by integrating kernels and pooling, which uniquely learns the most important features in the image. Bayesian logistic regression on the other hand, flattens high-dimensional images into a single vector, treats each pixel as an independent feature, and ultimately, ignores spatial context.

Both MLP and Bayesian logistic regression rely on the assumption that input features are independent, which is not fully satisfied in image data where neighboring pixels are correlated.

This partially explains the weaker performance of the Bayesian model and the limitations of the MLP. In contrast, the CNN better aligns with the data structure by explicitly modeling spatial dependencies, making its assumptions more appropriate for this project. While not all assumptions are strictly met, our findings are supported by models that bring to life meaningful insights.

There are several improvements to be explored for future research. To start, creating time-series models that identify patterns in temporal data can allow wildfire risk to be handled prematurely, reducing the risk of damage and the waste of critical resources and manpower.

For the Bayesian logistic regression, it could be effective to apply kernels from the CNN during pre-processing to isolate the most important features before training the model. It may also be beneficial to incorporate other numeric features drawn from satellite data (as previous studies have demonstrated).

Furthermore, this project could be extended by integrating satellite/mapping APIs to allow real-time predictions. By connecting the model to continuously updated data sources, the framework could be a more dynamic wildfire monitoring tool. Future work could also include opening the project to public contributions to improve model performance and expand geographic coverage. All in all, these ideas would only improve the scalability and real-world applicability of this preliminary environmental monitoring tool.

VI. BIBLIOGRAPHY

- [1] G. Charizanos and H. Demirhan, "Bayesian prediction of wildfire event probability using normalized difference vegetation index data from an Australian forest," *Ecological Informatics*, vol. 73, p. 101899, Mar. 2023, doi: <https://doi.org/10.1016/j.ecoinf.2022.101899>.
- [2] W. M. Dlamini, "Application of Bayesian networks for fire risk mapping using GIS and remote sensing data," *GeoJournal*, vol. 76, no. 3, pp. 283–296, May 2010, doi: <https://doi.org/10.1007/s10708-010-9362-x>.
- [3] P. Jain, S. C. P. Coogan, S. G. Subramanian, M. Crowley, S. Taylor, and M. D. Flannigan, "A review of ML applications in wildfire science and management," *Environmental Reviews*, vol. 28, no. 4, pp. 478-505, Jul. 2020, doi: <https://doi.org/10.1139/er-2020-0019>.
- [4] A. Bjånes, R. De La Fuente, and P. Mena, "A deep learning ensemble model for wildfire susceptibility mapping," *Ecological Informatics*, vol. 65, p. 101397, Nov. 2021, doi: <https://doi.org/10.1016/j.ecoinf.2021.101397>.
- [5] R. Kanwal, W. Rafaqat, M. Iqbal, and S. Weigu, "Data-Driven Approaches for Wildfire Mapping and Prediction Assessment Using a Convolutional Neural Network (CNN)," *Remote Sensing*, vol. 15, no. 21, p. 5099, Jan. 2023, doi: <https://doi.org/10.3390/rs15215099>.
- [6] M. Marjani, M. Mahdianpari, and F. Mohammadimanesh, "CNN-BiLSTM: A Novel Deep Learning Model for Near-Real-Time Daily Wildfire Spread Prediction," *Remote Sensing*, vol. 16, no. 8, p. 1467, Jan. 2024, doi: <https://doi.org/10.3390/rs16081467>.

VII. APPENDICES

A. Implementation Code

MLP	<pre># activation functions # https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/ def relu(z): return np.maximum(0, z)</pre>
-----	---

```

def relu_deriv(z):
    return (z > 0).astype(np.float32)

# clipping to introduce stability when the model gets trained
def sigmoid(z):
    z = np.clip(z, -50, 50)
    return 1.0 / (1.0 + np.exp(-z))

'''
Helper functions!
'''

# model performance to measure predicted to actual labels
def log_loss(y_true, p, eps=1e-7):
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y_true * np.log(p) + (1 - y_true) *
np.log(1 - p))

# binary classification accuracy score
def accuracy_score(y_true, p):
    yhat = (p >= 0.5).astype(int)
    return np.mean(yhat == y_true)

# step size
# accommodates the higher dimensionality of image data
eta = 0.001

# max iterations
epochs = 50

# hidden layer nodes
h = 32

# batch processing
batch_size = 128

n_train, d = X_train.shape

# initializing weights
W1 = np.random.randn(d, h).astype(np.float32) * np.sqrt(2.0 /
d)
b1 = np.zeros((1, h), dtype=np.float32)

W2 = np.random.randn(h, 1).astype(np.float32) * np.sqrt(2.0 /
h)
b2 = np.zeros((1, 1), dtype=np.float32)

# training loop
# 1 hidden layer neural network with mini-batch GD

'''
Methodology:
- randomize dataset ordering
- splitting data into batches
- perform forward and backward pass on each batch
- updating weights with gradient descent

```


	<pre> - evaluating performance on test AND validation sets ''' for epoch in range(epochs): # randomize ordering indices = np.random.permutation(n_train) X_train_rand = X_train[indices] y_train_rand = y_train[indices] # putting training data into batches for start in range(0, n_train, batch_size): end = start + batch_size Xb = X_train_rand[start:end] yb = y_train_rand[start:end] # forward pass operation z1 = Xb @ W1 + b1 a1 = relu(z1) # output layer z2 = a1 @ W2 + b2 p = sigmoid(z2) # backward pass operation m = Xb.shape[0] # output layer dz2 = (p - yb) / m dW2 = a1.T @ dz2 db2 = np.sum(dz2, axis=0, keepdims=True) # relu backprop dz1 = (dz2 @ W2.T) * relu_deriv(z1) dW1 = Xb.T @ dz1 db1 = np.sum(dz1, axis=0, keepdims=True) # updating parameter # gradient descent W2 -= eta * dW2 b2 -= eta * db2 W1 -= eta * dW1 b1 -= eta * db1 </pre>
CNN	<pre> # We expect 2 classes: 0 (no wildfire), 1 (wildfire) num_classes = len(class_names) img_dim = (256, 256) # Instantiating a Keras tensor inpx = Input(shape=(img_dim[0], img_dim[1], num_channels)) # Normalizing the data to be between 0 and 1 inpx = Rescaling(1./255)(inpx) # Instantiating first convolutional and pooling layer conv_layer = Conv2D(16, kernel_size=(3,3), strides=1, </pre>

	<pre> activation='relu')(inpx) pool_layer = MaxPooling2D(pool_size=(2,2))(conv_layer) # Instantiating second convolutional and pooling layer conv_layer2 = Conv2D(32, kernel_size=(3,3), strides=1, activation='relu')(pool_layer) pool_layer2 = MaxPooling2D(pool_size=(2,2))(conv_layer2) # Flattening the images for the hidden layer/output flat_G = Flatten()(pool_layer2) # Defining first hidden layer hid_layer = Dense(64, activation='relu')(flat_G) # Defining output layer out_layer = Dense(1, activation='sigmoid')(hid_layer) model = Model([inpx], out_layer) # Compile the model model.compile(optimizer=Adam(learning_rate=1e-3), loss=BinaryCrossentropy(), metrics=['accuracy', keras.metrics.TruePositives(name='tp'), keras.metrics.FalsePositives(name='fp'), keras.metrics.TrueNegatives(name='tn'), keras.metrics.FalseNegatives(name='fn')]) epochs = 10 # Training the model history = model.fit(train_dataset, batch_size=32, validation_data=valid_dataset, epochs=epochs) </pre>
Bayesian	<pre> library(brms) # Set default prior (use Bernoulli for binary classification) default_prior <- default_prior(label ~ ., # Use all independent collapsed pixels as features data = train_df, family = bernoulli("logit")) # Creating Bayesian logistic regression model brm_logit <- brm(label ~ ., family = bernoulli("logit"), data = train_df, chains = 4, </pre>

```
iter = 1000,  
warmup = 180,  
prior = default_prior)
```

```
summary(brm_logit)  
plot(brm_logit)
```

```
post_draws <- as.data.frame(brm_logit)
```